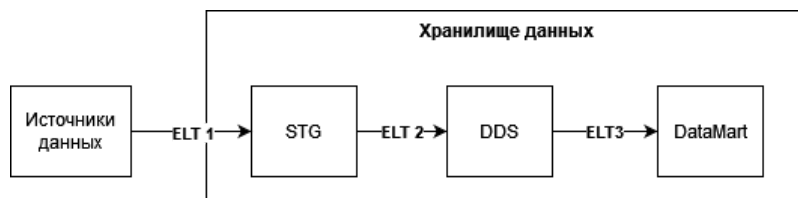


Выполнение тестового задания ДИП:КОД на инженера данных в “Газпром нефть”

Условие:

Необходимо реализовать загрузку данных в корпоративное хранилище в слой DDS, для моделирования слоя DDS использовать методологию Data Vault 2.0. Скрипт загрузки разработать на python. Данные загружать в БД PostgreSQL. (Если нет своей тестовой БД, инструкция по получению БД в Приложении 1).

Слои хранилища представлены на схеме:



Задача:

1. Спроектировать два слоя: STG и DDS;
2. Разработать потоки перекладки и трансформации данных: ELT 1 и ELT 2.

Тестовый источник

Как источник данных необходимо использовать REST-сервис

<https://jsonplaceholder.typicode.com/posts/>. В качестве бизнес-ключей использовать поля Id и userId.

Результат выполнения задания:

1. DDL скрипты по созданию таблиц в слоях STG и DDS;
 - a. Создание схем: schemas.sql
 - b. Создание таблицы STG: stg/posts.sql
 - c. Создание таблиц DDS: папка DDS
2. python скрипты для потоков ELT 1 и ELT 2 - в папке elt

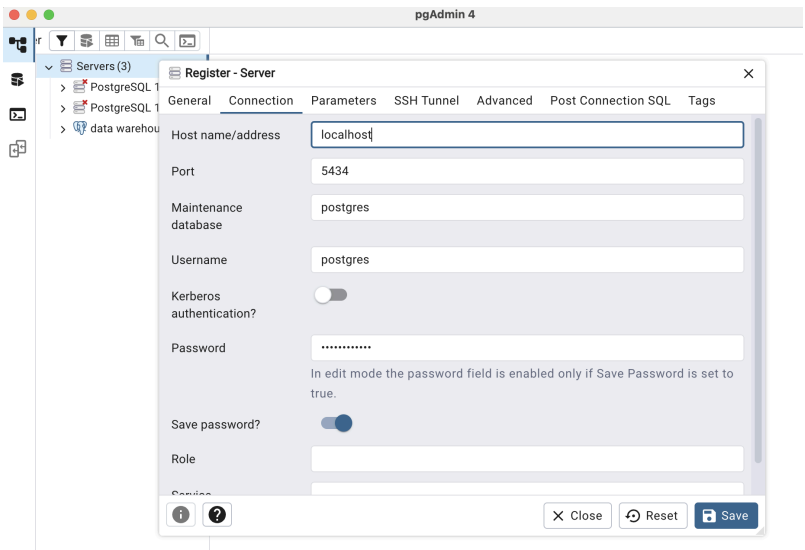
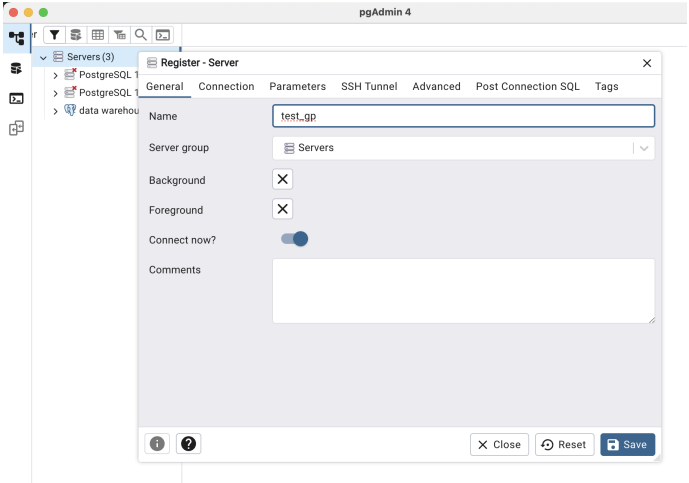
Выполнение:

1. Работа с БД

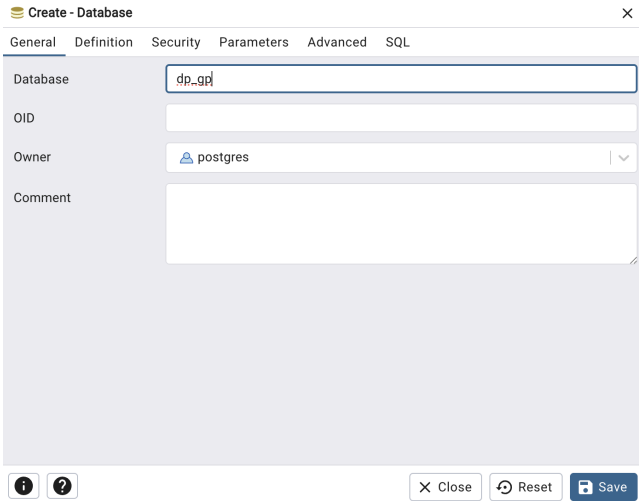
У меня уже стояли Docker Desktop и pgAdmin для работы с БД, поэтому я просто использовала уже существующий контейнер

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last st	Actions
☐	pg	e03580305559	postgres:1	5434:5432	0.06%	23 sec	   

Создала новый сервис test_gp



Создала БД db_gp в нем



2. STG слой

Предназначен для приема данных из источников. Данные хранятся здесь в том же виде, в каком они были получены

Создаю схему stg и таблицу posts (см. файл schemas.sql и stg/posts.sql)

3. DDS слой

По заданию нужно использовать методологию Data Vault 2.0.

Значит, делим данные на хабы, ссылки, спутники

1. Хабы - сущности, представляющие ключевые концепции/объекты (бизнес-ключи)

В нашем случае это Id и userId (по условию)

Структура состоит из полей хеш-ключа, бизнес-ключа (беру id из stg), даты загрузки и источника (везде jsonplaceholder)

2. Ссылки - связь между двумя или более хабами, определяет их отношения, моделирует транзакции и события внутри системы (например, “пост написан пользователем”)

Состоит из хеш-ключа, хеш-ключа пользователя, хеш-ключей хабов, дата загрузки, источник (везде jsonplaceholder)

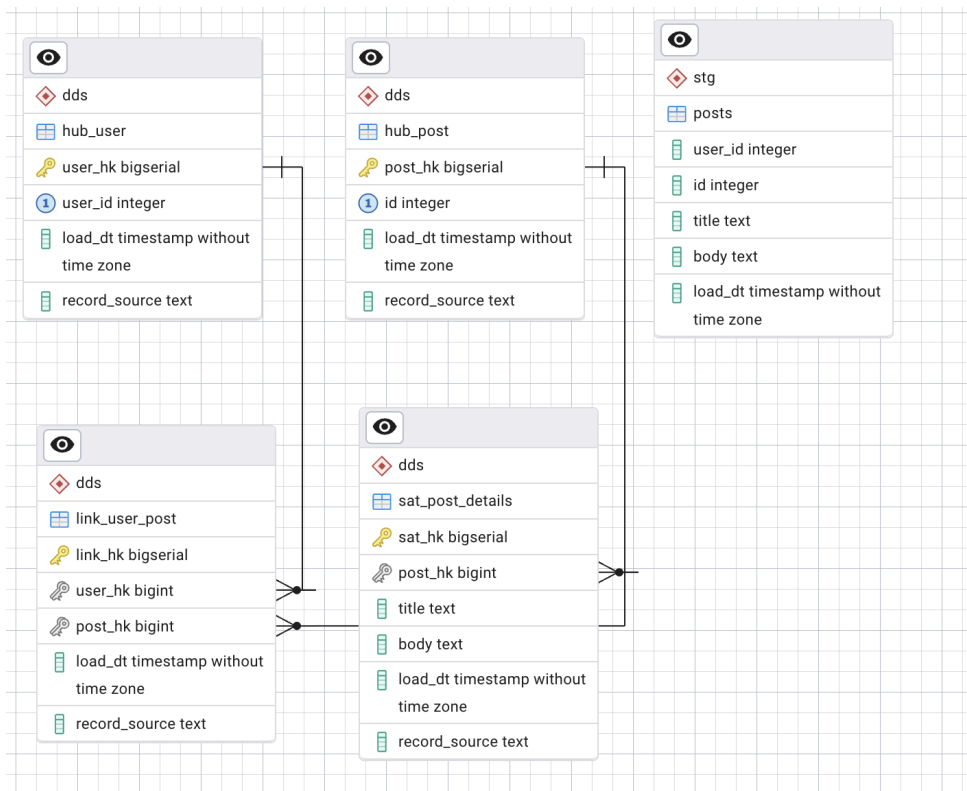
3. Спутники - таблица, содержащая описательные атрибуты бизнес-сущностей, которые могут изменяться со временем

В нашем случае это посты, хранятся атрибуты title и body.

Структура состоит из хеш-ключа, хеш-ключа хаба, описательных атрибутов, даты загрузки и источника (везде jsonplaceholder)

Создаю таблицы согласно описанным структурам (см. папку dds, названия файлов соответствуют сущностям)

Получаю такую схему:



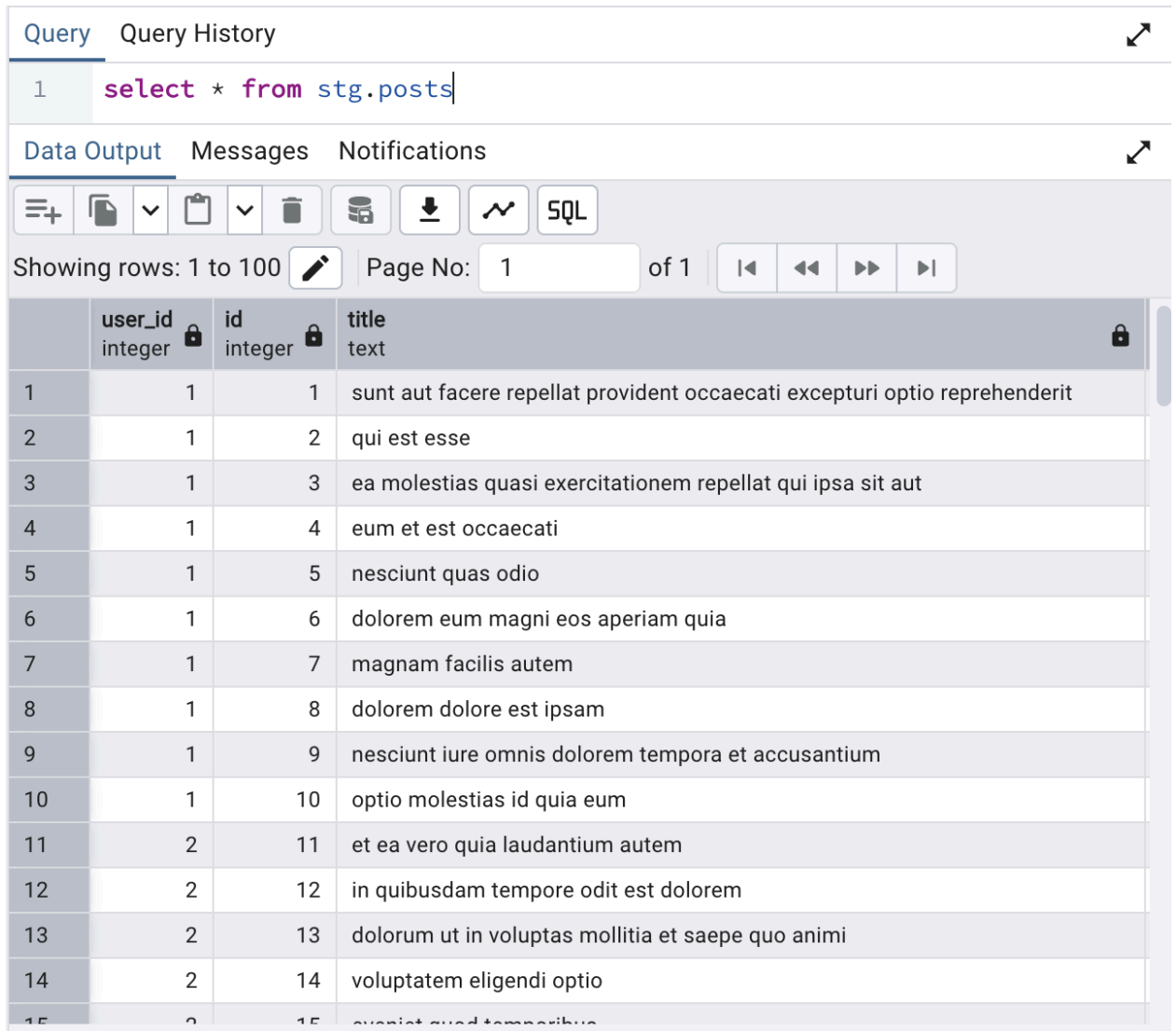
4. ELT 1

Извлекаем данные из источника, для этого использую REST API

<https://jsonplaceholder.typicode.com/posts/> и библиотеку requests и получаю ответ сервера в формате json. Далее каждая запись (с помощью цикла) вставлялась в stg.posts с помощью insert без каких-либо преобразований.

Результат:

```
test/venv/bin/python" "/Users/lena.arslanova/gp test/k.py"
Данные успешно загружены в stg.posts
```



The screenshot shows a SQL client interface with a query bar containing `select * from stg.posts`. Below the query bar, there are tabs for 'Query', 'Query History', 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, displaying a table with 15 rows. The table has columns: 'id' (integer), 'user_id' (integer), and 'title' (text). The data is as follows:

	user_id integer	id integer	title text
1	1	1	sunt aut facere repellat provident occaecati excepturi optio reprehenderit
2	1	2	qui est esse
3	1	3	ea molestias quasi exercitationem repellat qui ipsa sit aut
4	1	4	eum et est occaecati
5	1	5	nesciunt quas odio
6	1	6	dolorem eum magni eos aperiam quia
7	1	7	magnam facilis autem
8	1	8	dolorem dolore est ipsam
9	1	9	nesciunt iure omnis dolorem tempora et accusantium
10	1	10	optio molestias id quia eum
11	2	11	et ea vero quia laudantium autem
12	2	12	in quibusdam tempore odit est dolorem
13	2	13	dolorum ut in voluptas mollitia et saepe quo animi
14	2	14	voluptatem eligendi optio
15	2	15	eveniet quod temporibus

5. ELT 2

Далее используем stg.posts как источник, распределяем данные по сущностям (хабы, ссылки, спутники). Для этого использую SQL-запросы с сохранением бизнес-ключей(id, user_id) в хабах, проверкой на дубликаты через not exists и объединением таблиц для ссылочной сущности.

Результат:

```
(venv) lena.arslanova@MacBook-Air-Lena gp test % "/Users/lena.arslanova/gp
test/venv/bin/python" "/Users/lena.arslanova/gp test/elt2.py"
Данные успешно загружены в DDS
```

Query

Query History

1select * from dds.hub_user

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 10

Page No: 1 of 1

	user_hk [PK] bigint	user_id integer	load_dt timestamp without time zone	record_source text
1	11	1	2026-05-20 08:08:51.696071	jsonplaceholder
2	12	2	2026-05-20 08:08:51.696071	jsonplaceholder
3	13	3	2026-05-20 08:08:51.696071	jsonplaceholder
4	14	4	2026-05-20 08:08:51.696071	jsonplaceholder
5	15	5	2026-05-20 08:08:51.696071	jsonplaceholder
6	16	6	2026-05-20 08:08:51.696071	jsonplaceholder
7	17	7	2026-05-20 08:08:51.696071	jsonplaceholder
8	18	8	2026-05-20 08:08:51.696071	jsonplaceholder
9	19	9	2026-05-20 08:08:51.696071	jsonplaceholder
10	20	10	2026-05-20 08:08:51.696071	jsonplaceholder

Query

Query History

1

select * from dds.hub_post

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 100

Page No: 1

of 1

	post_hk [PK] bigint	id integer	load_dt timestamp without time zone	record_source text
1	101	55	2026-05-20 08:08:51.696071	jsonplaceholder
2	102	27	2026-05-20 08:08:51.696071	jsonplaceholder
3	103	23	2026-05-20 08:08:51.696071	jsonplaceholder
4	104	56	2026-05-20 08:08:51.696071	jsonplaceholder
5	105	91	2026-05-20 08:08:51.696071	jsonplaceholder
6	106	58	2026-05-20 08:08:51.696071	jsonplaceholder
7	107	8	2026-05-20 08:08:51.696071	jsonplaceholder
8	108	87	2026-05-20 08:08:51.696071	jsonplaceholder
9	109	74	2026-05-20 08:08:51.696071	jsonplaceholder
10	110	54	2026-05-20 08:08:51.696071	jsonplaceholder
11	111	29	2026-05-20 08:08:51.696071	jsonplaceholder
12	112	71	2026-05-20 08:08:51.696071	jsonplaceholder
13	113	68	2026-05-20 08:08:51.696071	jsonplaceholder
14	114	4	2026-05-20 08:08:51.696071	jsonplaceholder
15	115	34	2026-05-20 08:08:51.696071	jsonplaceholder

Query

Query History

1

select * from dds.link_user_post

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 100

Page No: 1 of 1

	link_hk [PK] bigint	user_hk bigint	post_hk bigint	load_dt timestamp without time zone	record_source text
1	1	18	112	2026-05-20 08:08:51.696071	jsonplaceholder
2	2	16	110	2026-05-20 08:08:51.696071	jsonplaceholder
3	3	19	124	2026-05-20 08:08:51.696071	jsonplaceholder
4	4	17	181	2026-05-20 08:08:51.696071	jsonplaceholder
5	5	11	125	2026-05-20 08:08:51.696071	jsonplaceholder
6	6	19	174	2026-05-20 08:08:51.696071	jsonplaceholder
7	7	14	115	2026-05-20 08:08:51.696071	jsonplaceholder
8	8	20	151	2026-05-20 08:08:51.696071	jsonplaceholder
9	9	11	165	2026-05-20 08:08:51.696071	jsonplaceholder
10	10	15	162	2026-05-20 08:08:51.696071	jsonplaceholder
11	11	15	127	2026-05-20 08:08:51.696071	jsonplaceholder
12	12	18	170	2026-05-20 08:08:51.696071	jsonplaceholder
13	13	17	123	2026-05-20 08:08:51.696071	jsonplaceholder
14	14	19	130	2026-05-20 08:08:51.696071	jsonplaceholder
15	15	17	113	2026-05-20 08:08:51.696071	jsonplaceholder

Query Query History			
1 select * from dds.sat_post_details			
Data Output Messages Notifications			
Showing rows: 1 to 100 Page No: 1 of 1			
	sat_hk [PK] bigint	post_hk bigint	title text
1	1	196	sunt aut facere repellat provident occaecati excepturi optio reprehenderit
2	2	147	qui est esse
3	3	190	ea molestias quasi exercitationem repellat qui ipsa sit aut
4	4	114	eum et est occaecati
5	5	198	nesciunt quas odio
6	6	128	dolorem eum magni eos aperiam quia
7	7	166	magnam facilis autem
8	8	107	dolorem dolore est ipsam
9	9	165	nesciunt iure omnis dolorem tempora et accusantium
10	10	125	optio molestias id quia eum
11	11	154	et ea vero quia laudantium autem
12	12	173	in quibusdam tempore odit est dolorem
13	13	145	dolorum ut in voluptas mollitia et saepe quo animi
14	14	141	voluptatem eligendi optio
15	15	169	eveniet quod temporibus

6. Проверка

- При верной работе всех запросов количество уникальных пользователей в stg.posts должно равняться количеству пользователей в dds.hub_users

Query Query History	
1 select count(distinct user_id) from stg.posts	
Data Output Messages Notifications	
Showing rows: 1 to 1 Page No: 1 of 1	
	count bigint
1	10

Query Query History	
1 select count(user_id) from dds.hub_user	
Data Output Messages Notifications	
Showing rows: 1 to 1 Page No: 1 of 1	
	count bigint
1	10

- Аналогично с постами

Query Query History

```
1 select count(id) from dds.hub_post
```

Data Output Messages Notifications

	count bigint
1	100

Query Query History

```
1 select count(distinct id) from stg.posts
```

Data Output Messages Notifications

	count bigint
1	100

- Количество строк в stg.posts должно равняться количеству строк в dds.link_user_post

Query Query History

```
1 select count(*) from dds.link_user_post
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	count bigint
1	100

```
1 select count(*) from stg.posts
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	count bigint
1	100

- У каждого поста есть набор атрибутов, значит количество строк в stg.posts должно равняться количеству строк в dds.sat_post_details
См. прошлый пункт для количества строк в stg.posts

Query Query History

```
1 select count(*) from dds.sat_post_details
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	count bigint
1	100

- Проверим на дубликаты:

Query Query History

```
1 select user_id, count(*)
2 from dds.hub_user
3 group by user_id
4 having count(*) > 1
```

Data Output Messages Notifications

user_id integer	count bigint
--------------------	-----------------

Query Query History

```
1 select id, count(*)
2 from dds.hub_post
3 group by id
4 having count(*) > 1
```

Data Output Messages Notifications

id integer	count bigint
---------------	-----------------

- ничего не вывелось,
дубликатов нет

Значит, все выполнено верно

